

INTRODUÇÃO C++

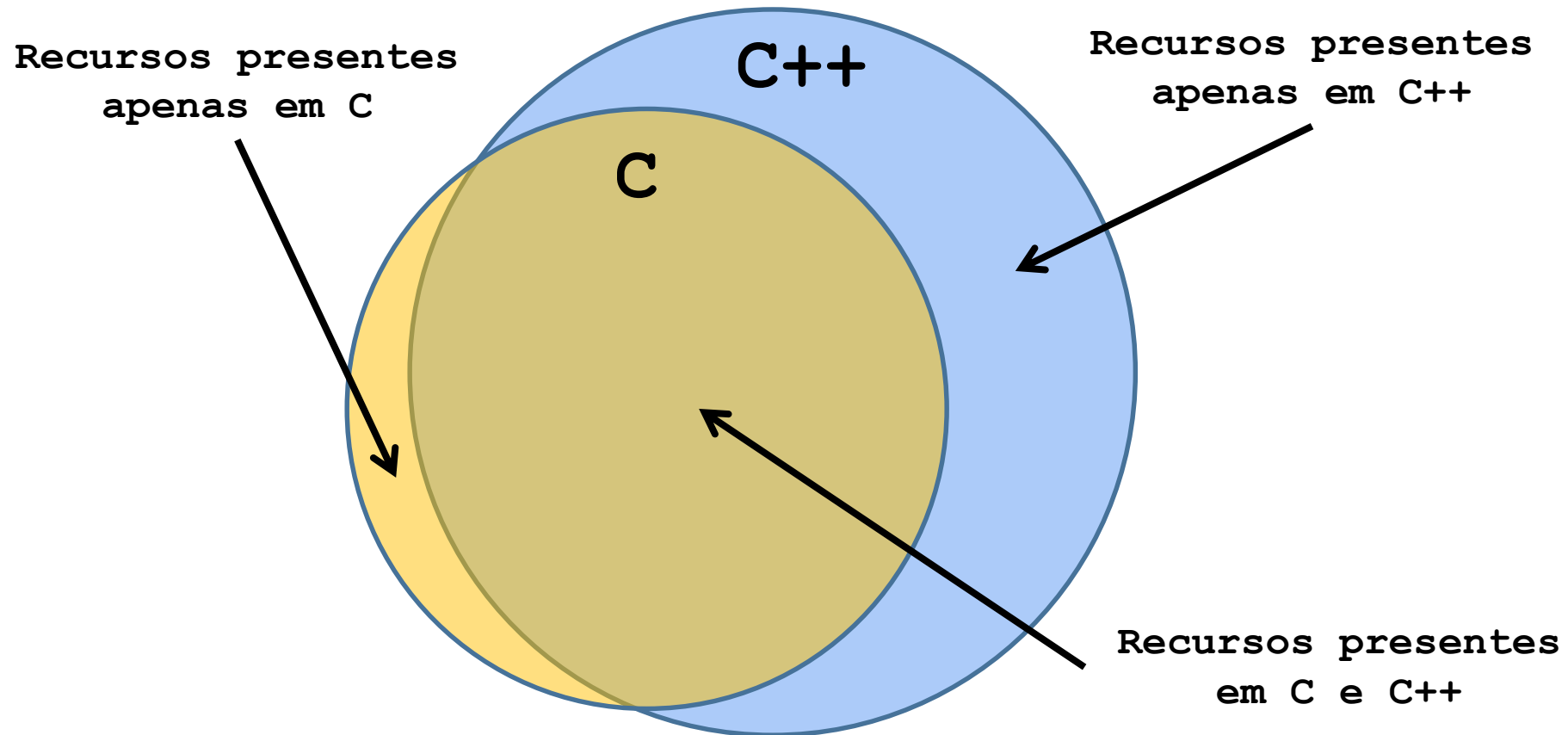
Prof. André Backes

Linguagem C++

- Linguagem de propósito geral criada por Bjarne Stroustrup
- Originalmente criada como uma extensão da linguagem C
 - “C com Classes”
- Suporta programação procedimental e orientada a objetos

C é um subconjunto do C++?

- Não exatamente...



C é um subconjunto do C++?

- Foco nas diferenças do C e C++ na parte procedural
 - O que funciona igual a linguagem C?
 - O que o C++ tem de novidade?

```
#include <stdio.h>

int main(){
    printf("Hello World!\n");

    return 0;
}
```

```
#include <iostream>

using namespace std;

int main(){
    cout << "Hello World!" << endl;

    return 0;
}
```

Trabalhando com namespace

- Um **namespace** declara um escopo contendo um conjunto de objetos relacionados
 - Permite agrupar entidades como classes, objetos e funções sob um nome
 - Divide o escopo global em "subescopos", cada um com seu próprio nome

```
namespace nome_namespace{  
    ///declarações  
}
```

Trabalhando com namespace

- O **namespace std** contém as funções da biblioteca padrão
 - Para usar um elemento de um **namespace** usamos o operador **::**
 - Outra forma é usar a palavra-chave **using** para introduzir o namespace

```
#include <iostream>
using namespace std;

int main(){
    cout << "Hello World!";

    return 0;
}

#include <iostream>

int main(){
    std::cout << "Hello World!";

    return 0;
}
```

Trabalhando com namespace

- Um namespace permite reutilizar o mesmo nome de função para ações diferentes

```
#include <iostream>
namespace space1{
    void imprime(){
        std::cout << "Namespace 1";
    }
}

namespace space2{
    void imprime(){
        std::cout << "Namespace 2";
    }
}

int main(){
    space1::imprime();
    space2::imprime();

    return 0;
}
```

Escrevendo e lendo dados em C++

- Na linguagem C usamos a função **printf()** para escrever dados.
- Na linguagem C++ utilizamos o objeto **cout**, contido na biblioteca `iostream`, namespace `std`
- Fora geral:

```
cout << dados;
```

- Podemos também exibir diversas variáveis em sequência:

```
cout << dados_1 << dados_2 << ... << dados_N;
```


Escrevendo e lendo dados em C++

- O **endl** equivale ao '\n'
 - Também força a liberação do buffer de dados

```
#include <iostream>
using namespace std;

int main(){
    int x = 12;
    float y = 5.0;
    //Exibe um texto
    cout << "Esse texto sera escrito na tela\n";
    //Exibe um texto e uma variável
    cout << "x = " << x << endl;
    //Exibe um um valor inteiro e outro real
    cout << x << y << endl;
    //Adicionando espaço entre os valores
    cout << x << " " << y << endl;

    return 0;
}
```

Escrevendo e lendo dados em C++

- O objeto **cout** possui vários métodos que permitem manipular melhor os dados de saída

```
int main(){
    float y = 5.17;
    //Exibe y
    cout << y << endl;
    //Exibe y com largura 10
    cout.width(10);
    cout << y << endl;
    //Exibe y com largura 10, completando com x
    cout.width(10);
    cout.fill('x');
    cout << y << endl;
    //Exibe y com precisão 15
    cout.precision(15);
    cout << y << endl;

    return 0;
}
```

Escrevendo e lendo dados em C++

- Na linguagem C usamos a função **scanf()** para ler dados.
- Na linguagem C++ utilizamos o objeto **cin**, contido na biblioteca `iostream`, namespace `std`
- Fora geral:

```
cin >> variavel;
```

- Podemos também exibir diversas variáveis em sequência:

```
cin >> var_1 >> var_2 >> ... >> var_N;
```

Escrevendo e lendo dados em C++

- As variáveis serão preenchidas com seus respectivos dados seguindo a ordem especificada, da esquerda para a direita

```
#include <iostream>
using namespace std;

int main() {
    cout << "Digite a idade: ";
    //Lê um int
    int idade;
    cin >> idade;

    cout << "Digite o peso e a altura: ";
    //Lê dois double
    double peso, altura;
    cin >> peso >> altura;

    return 0;
}
```

Escrevendo e lendo dados em C++

- O objeto **cin** possui vários métodos que permitem manipular melhor os dados de entrada

```
int main() {
    double x;
    cout << "Digite um valor no formato double: ";
    cin >> x;
    if(cin.fail()){// Erro na extração?
        cout << "Valor invalido!" << endl;
        // limpa o estado de erro do buffer
        cin.clear();
        // descarta o restante do buffer
        cin.ignore(32767, '\n');
    }else{
        cout << "Valor digitado: " << x;
    }

    return 0;
}
```

Variáveis e Tipos

- A declaração de uma variável segue a mesma forma, e restrições, da linguagem C
 - Estão presentes todos os tipos básicos do C
- Tipo booleano
 - Não existe na linguagem ANSI-C

```
int main() {  
  
    bool v1 = false;  
    bool v2 = true;  
  
    return 0;  
}
```

Variáveis e Tipos

- Ponto flutuante padrão IEEE 754
 - Assim como o padrão C99, o C++ possui suporte à representação de valores em ponto flutuante de acordo com o padrão IEEE 754
 - É o padrão seguido por quase todas as máquinas modernas
 - Isso inclui suporte a valores infinito e “não um é número” (NaN), no caso de um resultado indefinido ou que não pode ser representado com precisão

Variáveis e Tipos

- Funções para testar se o valor é infinito ou NaN
 - `isinf()`: testa se um valor é infinito (true) ou não (false).
 - `isnan()`: testa se um valor é NaN (true) ou não (false).

```
int main(){
    double zero = 0.0;
    double v1 = 5.0 / zero; // infinito positivo
    cout << "V1: " << v1 << endl;

    double v2 = -5.0 / zero; // infinito negativo
    cout << "V2: " << v2 << endl;

    double v3 = sqrt(-1); // não um é número
    cout << "V3: " << v3 << endl;

    cout << isinf(zero) << endl; // 0
    cout << isinf(v2) << endl; // 1
    cout << isnan(v3) << endl; // 1

    return 0;
}
```


Variáveis e Tipos

- O padrão C++11 suporta novos tipos inteiros para melhorar a portabilidade de programas
 - Tipos inteiros de largura exata
 - N bits em todos os sistemas: `int8_t`, `int16_t`, `int32_t` e `int64_t`
 - Tipos inteiros de largura mínima
 - Possuem pelo menos N bits: `int_least8_t`, `int_least16_t`, `int_least32_t` e `int_least64_t`
 - Tipos inteiros rápidos
 - Tipos mais rápidos disponíveis com pelo menos N bits: `int_fast8_t`, `int_fast16_t`, `int_fast32_t` e `int_fast64_t`

Variáveis e Tipos

- Tipo inteiro de largura máxima
 - Capaz de representar todos os demais tipos inteiros: `intmax_t`
- Ponteiro de inteiro
 - Garante poder armazenar um ponteiro para inteiro: `intptr_t`
- A versão unsigned é obtida colocando-se o prefixo **u** no tipo
 - `int8_t`: inteiro 8 bits com sinal
 - `uint8_t`: inteiro 8 bits sem sinal

Variáveis e Tipos

- Deduzindo o tipo da variável
 - Antes do padrão C++11, a palavra chave auto se referia a uma classe de armazenamento de variáveis
 - Agora podemos usar essa palavra chave no lugar do tipo da variável inicializada para informar que o tipo da variável será o mesmo do valor da sua inicialização

```
int main() {  
  
    double N1 = 5.25;  
    auto N2 = 5.25;  
  
    cout << sizeof(N1) << endl;  
    cout << sizeof(N2) << endl;  
  
    return 0;  
}
```

C vs C++ | Inicializando uma variável

- C++ possui três formas diferentes de inicializar uma variável:
 - Inicialização por cópia, operador de atribuição

```
int x = 5;
```

- Inicialização direta, com parênteses ()

```
int Y(5);
```

- Inicialização uniforme (padrão C++11), com chaves {}:

```
int z{5};
```

C vs C++

- A linguagem C++ possui os mesmo operadores que a linguagem C
 - Aritméticos, relacionais, lógicos ...
- Os comandos condicionais funcionam da mesma forma
 - if-else, switch, break, ...
- Assim como os comandos de repetição
 - while, for e do-while
- E os arrays (vetores e matrizes)

C vs C++ | Comando for

- Variáveis podem ser declaradas na inicialização

Linguagem C

```
int V[5] = {10 ,20 ,30 ,40 ,50};  
int i;  
for(i = 0; i < 5; i++)  
    printf("%d\n",V[i]);
```

Linguagem C++

```
int V[5] = {10 ,20 ,30 ,40 ,50};  
  
for(int i = 0; i < 5; i++)  
    cout << V[i] << "\n";
```

- Pode ser usado para processar uma lista de valores
 - Os valores da lista são copiados para a variável especificada

for tradicional

```
int V[5] = {10 ,20 ,30 ,40 ,50};  
  
for(int i = 0; i < 5; i++)  
    cout << V[i] << "\n";
```

for para listas de valores

```
int V[5] = {10 ,20 ,30 ,40 ,50};  
  
for(int x: V)  
    cout << x << "\n";
```

C vs C++ | struct

- Estruturas funcionam iguais, mas com algumas facilidades
 - Não é necessário usar a palavra struct antes do tipo da nova variável declarada

Linguagem C

```
struct cadastro{  
    char nome[50];  
    int idade;  
    char rua[100];  
    int numero;  
};  
  
struct cadastro c;
```

Linguagem C++

```
struct cadastro{  
    char nome[50];  
    int idade;  
    char rua[100];  
    int numero;  
};  
  
cadastro c;
```

C vs C++ | struct

- Estruturas anônimas
 - Podemos omitir o nome da struct se sua definição for acompanhada da declaração de variáveis
- O padrão C++11 permite fazer a atribuição de um conjunto de valores usando uma lista de inicialização

```
struct {  
    string nome;  
    int idade;  
    string rua;  
    int numero;  
} cad1, cad2;  
  
//Inicialização tradicional  
cadastro c1 = {"Carlos", 18, "Avenida Brasil", 1082 };  
  
//Inicialização uniforme (padrão C++11)  
cadastro c2 {"Carlos", 18, "Avenida Brasil", 1082 };  
  
//Atribuição  
cadastro cad;  
cad = {"John", 38, "Avenida 1", 81};
```


C vs C++ | struct

- Nos padrões C++11 e C++14 é possível definir valores iniciais para cada campo da struct

```
using namespace std;

struct ponto{
    int x = 10;
    int y {20};
};

int main(){

    ponto p;

    cout << p.x << endl;
    cout << p.y << endl;

    return 0;
}
```

Material Complementar

- Aula 01 - Introdução
 - <https://youtu.be/B5ude60bbe0>
- Aula 02 - Trabalhando com namespace
 - <https://youtu.be/7h7qKFXV8LY>
- Aula 03 - Escrevendo e lendo dados em C++
 - <https://youtu.be/5GbzRd3vLUU>
- Aula 04 - Variáveis e Tipos
 - <https://youtu.be/dALXwxoXzYo>
- Aula 05 - C vs C++: operadores, condicional, repetição e array
 - <https://youtu.be/5cQUIJauNQs>
- Aula 06 - C vs C++: struct e typedef
 - <https://youtu.be/NEZpDGqJCUC>